

```
In [1]: import torch
import numpy as np
```

```
In [2]: # 从列表数据直接生成和初始化张量
data = [[1, 2],[3, 4]]
x_data = torch.tensor(data)
```

```
In [3]: x_data
```

```
Out[3]: tensor([[1, 2],
               [3, 4]])
```

```
In [4]: # 从Numpy数组直接生成
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
```

```
In [5]: x_np
```

```
Out[5]: tensor([[1, 2],
               [3, 4]], dtype=torch.int32)
```

```
In [6]: #从另外一个张量tensor生成
x_ones = torch.ones_like(x_data) # 大小和数据类型都与x_data保持一致，数据本身修改为单位
print(f"Ones Tensor: \n {x_ones} \n")
```

```
Ones Tensor:
tensor([[1, 1],
        [1, 1]])
```

```
In [7]: # 自定义张量的形状，然后通过随机函数或者常数函数生成：
shape = (2,3,)
rand_tensor = torch.rand(shape)
ones_tensor = torch.ones(shape)
zeros_tensor = torch.zeros(shape)

print(f"Random Tensor: \n {rand_tensor} \n")
print(f"Ones Tensor: \n {ones_tensor} \n")
print(f"Zeros Tensor: \n {zeros_tensor}")
```

```
Random Tensor:
tensor([[0.8357, 0.4768, 0.6774],
        [0.4075, 0.6985, 0.2727]])
```

```
Ones Tensor:
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

```
Zeros Tensor:
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

```
In [8]: # 张量的属性
```

```

tensor = torch.rand(3,4)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")

```

Shape of tensor: torch.Size([3, 4])
 Datatype of tensor: torch.float32
 Device tensor is stored on: cpu

```

In [9]: # 将张量转移至cuda设备上
        if torch.accelerator.is_available():
            tensor = tensor.to(torch.accelerator.current_accelerator())

```

```

In [10]: print(f"Device tensor is stored on: {tensor.device}")

```

Device tensor is stored on: cuda:0

```

In [11]: #从另外一个张量tensor生成, 强制修改dtype
        x_rand = torch.rand_like(x_data, dtype=torch.float) # 大小一致, 但数据类型被强制修改为
        print(f"Random Tensor: \n {x_rand} \n")

```

Random Tensor:
 tensor([[0.5543, 0.8467],
 [0.9253, 0.3190]])

```

In [12]: # tensor的切片和索引
        tensor = torch.ones(4, 4)
        print(f"First row: {tensor[0]}")
        print(f"First column: {tensor[:, 0]}")
        print(f>Last column: {tensor[..., -1]}")

```

First row: tensor([1., 1., 1., 1.])
 First column: tensor([1., 1., 1., 1.])
 Last column: tensor([1., 1., 1., 1.])

```

In [13]: tensor[:,1] = 0
        print(tensor)

```

tensor([[1., 0., 1., 1.],
 [1., 0., 1., 1.],
 [1., 0., 1., 1.],
 [1., 0., 1., 1.]])

```

In [16]: # 张量的合并
        t1 = torch.cat([tensor, tensor, tensor], dim=1)
        print(t1)

```

tensor([[1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
 [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
 [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.],
 [1., 0., 1., 1., 1., 0., 1., 1., 1., 0., 1., 1.]])

```

In [17]: # 张量的矩阵运算
        y1 = tensor @ tensor.T
        y2 = tensor.matmul(tensor.T)

```

```

y3 = torch.rand_like(y1)
torch.matmul(tensor, tensor.T, out=y3)
print(f"y1:={y1},y2:={y2},y3:={y3}")

```

```

y1:=tensor([[3., 3., 3., 3.],
            [3., 3., 3., 3.],
            [3., 3., 3., 3.],
            [3., 3., 3., 3.]])y2:=tensor([[3., 3., 3., 3.],
            [3., 3., 3., 3.],
            [3., 3., 3., 3.],
            [3., 3., 3., 3.]])y3:=tensor([[3., 3., 3., 3.],
            [3., 3., 3., 3.],
            [3., 3., 3., 3.],
            [3., 3., 3., 3.]])

```

```

In [18]: # 张量的算术运算
# 矩阵的算数乘, 每个标量对应相乘;
z1 = tensor * tensor
z2 = tensor.mul(tensor)

z3 = torch.rand_like(tensor)
torch.mul(tensor, tensor, out=z3)
print(f"z1={z1},z2={z2},z3={z3}")

```

```

z1=tensor([[1., 0., 1., 1.],
            [1., 0., 1., 1.],
            [1., 0., 1., 1.],
            [1., 0., 1., 1.]])z2=tensor([[1., 0., 1., 1.],
            [1., 0., 1., 1.],
            [1., 0., 1., 1.],
            [1., 0., 1., 1.]])z3=tensor([[1., 0., 1., 1.],
            [1., 0., 1., 1.],
            [1., 0., 1., 1.],
            [1., 0., 1., 1.]])

```

```

In [19]: # 将张量的各个标量加起来, 形成一个算数值
agg = tensor.sum()
agg_item = agg.item()
print(agg,type(agg))
print(agg_item, type(agg_item))

```

```

tensor(12.) <class 'torch.Tensor'>
12.0 <class 'float'>

```

```

In [20]: # in-place运算
y=torch.tensor([[1., 0., 1., 1.],
                [1., 0., 1., 1.],
                [1., 0., 1., 1.],
                [1., 0., 1., 1.]])
print(f"{tensor} \n")
tensor.add_(5) # 加法, 自加5
print(tensor)
print(f"{tensor} \n")
tensor.copy_(y) # 复制到自己
print(tensor)
print(f"{tensor} \n") # 自转置

```

```
tensor.t_()
print(tensor)
```

```
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

```
tensor([[6., 5., 6., 6.],
        [6., 5., 6., 6.],
        [6., 5., 6., 6.],
        [6., 5., 6., 6.]])
```

```
tensor([[6., 5., 6., 6.],
        [6., 5., 6., 6.],
        [6., 5., 6., 6.],
        [6., 5., 6., 6.]])
```

```
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

```
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

```
tensor([[1., 1., 1., 1.],
        [0., 0., 0., 0.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
```

In []:

In []:

```
In [ ]: points_cpu=torch.tensor([[4.0,1.0],[5.3,3.0],[2.0,1.0]],device='cpu')
```

```
In [ ]: points_cpu
```

```
In [ ]: points_gpu=torch.tensor([[4.0,1.0],[5.3,3.0],[2.0,1.0]],device='cuda')
```

```
In [ ]: points_gpu
```

In []:

In []: